Branch Coverage Analysis Report (self_storage.py)

Ryona Aditree Ghosh
PES1UG24AM233
ryonahh/L6_L7_51_self_storage

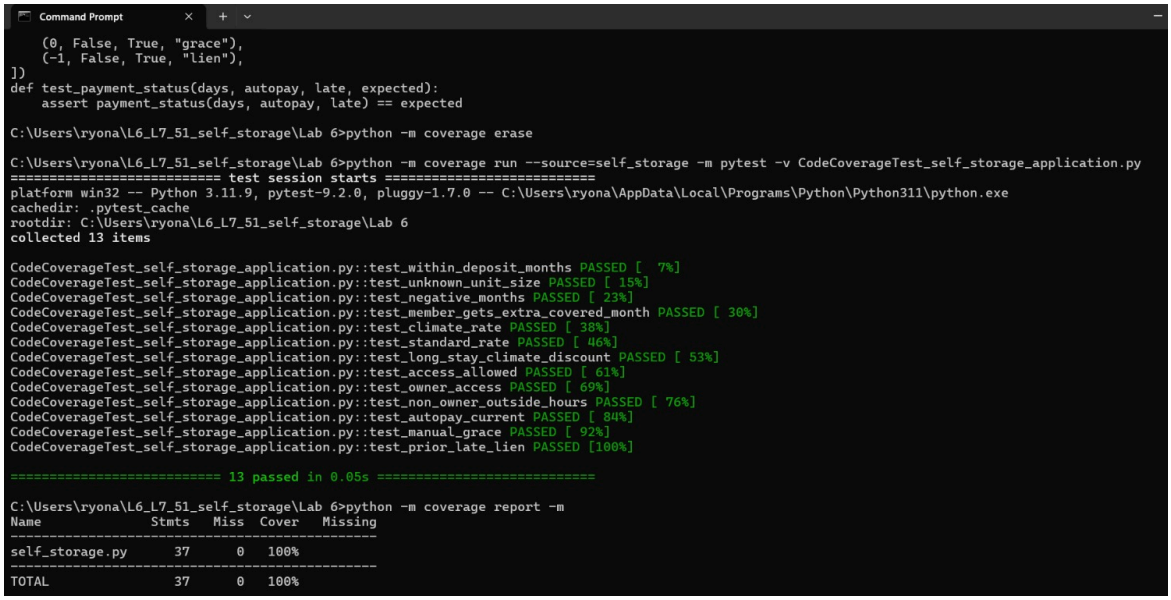
1. Objective and setup

This report measures branch coverage of self_storage.py. A suite with 100% line coverage still left one branch untested; the final suite of 40 tests covers all 26 branches (100%), above the 90% target.

Line coverage asks whether a line ran. Branch coverage asks whether every if and elif went both ways, true and false. The module has 13 decisions, so 26 branches: 7 decisions in rent_due, 2 in access_allowed and 4 in payment_status. A compound condition such as hour < 6 or hour > 22 counts as one decision. Tests ran on Windows with Python 3.11.9, pytest 9.2.0 and coverage 7.17.0, using coverage run --branch. All code and screenshots are in the Lab 6 folder of ryonahh/L6_L7_51_self_storage

2. Starting point: 100% line coverage, 98% branch coverage

The 13-test code-coverage suite reaches 100% of lines, but with --branch one branch of 26 is only partly covered, giving 98%.



The Missing entry 21->23 means the jump from line 21 to line 23 never happened. Line 21 is if is_climate: inside the long-stay discount. The earlier suite only ran that discount for a climate-controlled unit, so the false case was never taken. A non-climate unit with more than 6 extra months, charged 80 − 10 = 70 per month, closes the gap.

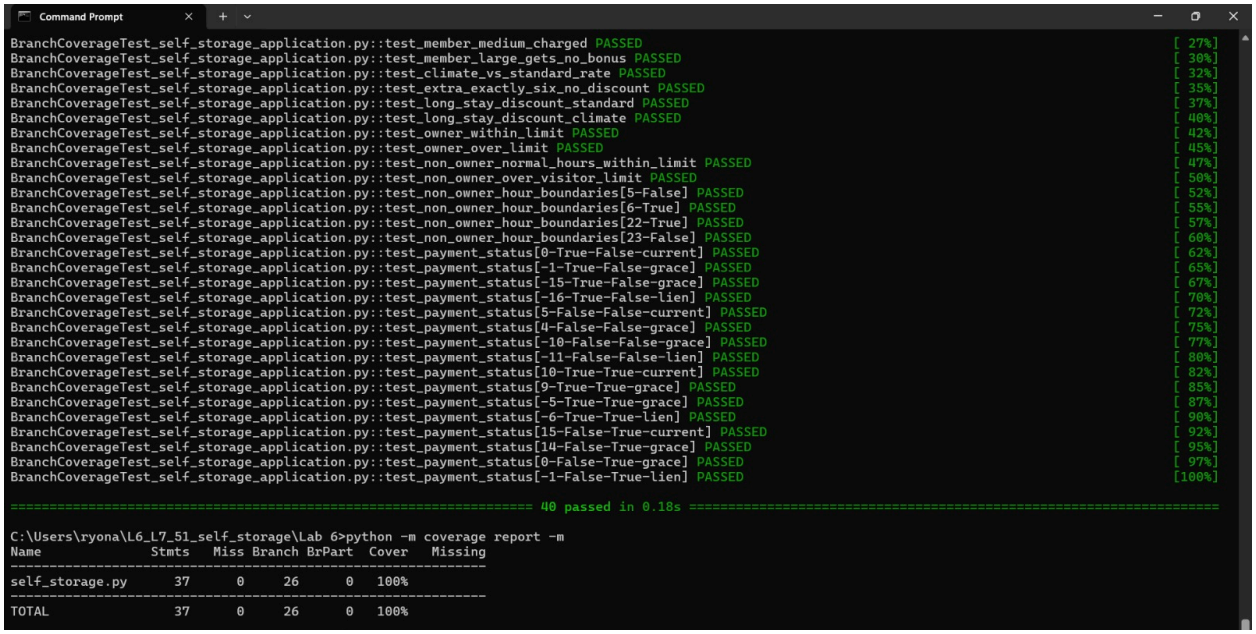
3. Test cases added

BranchCoverageTest_self_storage_application.py has 40 test cases. Every decision is exercised both ways, with extra cases on each boundary, since off-by-one errors usually live there.

- **rent_due (16 cases):** both ValueError paths, 0 months, the deposit boundary for small, medium and large units, the member bonus for each size (large gets none), climate and standard rates, and the long-stay discount on both. Six extra months costs 480 and seven costs 490 for a small non-climate unit, which shows the discount starts above 6.
- **access_allowed (8 cases):** an owner at 4 and 5 visitors, a non-owner at 2 and 3 visitors, and hours 5, 6, 22 and 23.
- **payment_status (16 cases):** all four combinations of autopay and prior late payment. Each is tested at its cutoff (current), one day below it and 15 days below it (grace), and 16 days below it (lien). The cutoff is 0 for autopay, 5 for manual, plus 10 with a prior late payment.

4. Final result: 100% branch coverage

All 40 tests pass, all 26 branches are taken both ways and none is partial, so branch coverage is 100% against the 90% target. Coverage with --branch moved from 32% (starter tests) to 98% (code-coverage suite) to 100% (this suite).



5. Analysis

Line coverage hid the gap. At line 21 the code has an if and no else, so when is_climate is false execution skips straight to line 23. No line goes missing, so the line report stayed at 100% while branch coverage showed 98% and named the exact jump, 21->23. One extra test (small unit, 8 months, no climate control) fixed it.

Boundary testing is where branch tests pay off. Operators such as <=, >= and > are easy to get off by one. Testing values just inside and outside each threshold proves each comparison flips where the rule says it should.

100% branch coverage is still not proof of correctness. Compound conditions count as one decision, branch coverage does not check combinations of decisions across rent_due, and the expected values come from the module's current rules, so a mistake in those rules would be locked in rather than caught. Even so, for finding untested decisions, branch coverage is the stronger measure: it exposes a gap that line coverage could not.

6. Commits and conclusion

- 90ea460 – Add branch coverage test cases (the test file)
- e5181fc – Add tests for branch coverage analysis (the screenshot)

Branch coverage of self_storage.py went from 32% (starter tests) to 98% (line-coverage suite) to 100% (40-test branch suite), meeting the 90% requirement. It also exposed the one decision, 21->23, that a 100% line-coverage report had missed.